

LAB 11. PHÂN QUYỀN CHỨC NĂNG THEO VAI TRÒ

MỤC TIÊU BÀI THỰC HÀNH

Kết thúc Lab này, sinh viên phải đạt được:

1. Hiểu được sự khác biệt giữa **Authentication** (Xác thực người dùng - Lab 10) và **Authorization** (Phân quyền người dùng - Lab 11).
2. Cập nhật CSDL để phân loại người dùng (Thêm cột role).
3. Tạo và cấu hình **Policies** cho Model Product.
4. Áp dụng nguyên tắc "Bảo vệ kép" (Double Protection): Khóa các hành động nhạy cảm (Thêm/Sửa/Xóa) ở cả tầng giao diện (Blade) và tầng xử lý logic (Controller).

HƯỚNG DẪN THỰC HÀNH

Bước 1: Cập nhật CSDL để phân loại vai trò (Role)

Hệ thống hiện tại có người dùng đăng nhập nhưng chưa phân biệt ai là Admin, ai là Khách thường. Ta cần thêm cột role vào bảng users.

1. Mở Terminal và chạy lệnh tạo Migration mới:

```
php artisan make:migration add_role_to_users_table --table=users
```

2. Mở file Migration vừa sinh ra trong thư mục database/migrations/, nội dung:

```
public function up(): void
{
    Schema::table('users', function (Blueprint $table) {
        // Mặc định mọi tài khoản tạo mới đều là 'user' (người dùng thường)
        // Đặt sau cột email cho dễ nhìn trong DB
        $table->string('role')->default('user')->after('email');
    });
}

public function down(): void
{
    Schema::table('users', function (Blueprint $table) {
        $table->dropColumn('role');
    });
}
```

3. Chạy lệnh áp dụng thay đổi vào CSDL:

```
php artisan migrate
```

4. Mở file app/Models/User.php, thêm cột role vào mảng \$fillable để cho phép lưu dữ liệu:

```
protected $fillable = [  
    'name',  
    'email',  
    'password',  
    'phone_number', // (Từ lab 10)  
    'role',          // THÊM DÒNG NÀY  
];
```

Bước 2: Chuẩn bị Dữ liệu Kiểm thử (Test Data)

Để có thể test việc phân quyền, chúng ta cần ít nhất 1 tài khoản Admin và 1 tài khoản User.

1. Mở công cụ quản lý CSDL (phpMyAdmin, TablePlus, DataGrip,...).
2. Truy cập vào bảng users.
3. Chỉnh sửa (Edit) trực tiếp dữ liệu của 1 tài khoản bất kỳ mà bạn muốn làm quản trị viên. Sửa giá trị ở cột role từ user thành admin.
4. Giữ nguyên 1 tài khoản khác với role là user.

Ghi nhớ Email và Mật khẩu của 2 tài khoản này để test luồng.

Bước 3: Khởi tạo và định nghĩa Policy cho Product

Policy giống như một luật quy định ai được phép làm gì trên một Model cụ thể.

1. Chạy lệnh tạo Policy trên Terminal:

```
php artisan make:policy ProductPolicy --model=Product
```

2. Mở file app/Policies/ProductPolicy.php và định nghĩa các luật. Yêu cầu bài toán:
Ai cũng được xem, nhưng chỉ Admin mới được Thêm/Sửa/Xóa.

```
namespace App\Policies;  
  
use App\Models\Product;  
use App\Models\User;
```

```
class ProductPolicy
{
    // Quyền xem danh sách: Mọi User đăng nhập đều được phép
    public function viewAny(User $user): bool
    {
        return true;
    }

    // Quyền xem chi tiết: Mọi User đăng nhập đều được phép
    public function view(User $user, Product $product): bool
    {
        return true;
    }

    // Quyền thêm mới: Chỉ tài khoản có role là 'admin'
    public function create(User $user): bool
    {
        return $user->role === 'admin';
    }

    // Quyền cập nhật (Sửa): Chỉ tài khoản có role là 'admin'
    public function update(User $user, Product $product): bool
    {
        return $user->role === 'admin';
    }

    // Quyền xóa: Chỉ tài khoản có role là 'admin'
    public function delete(User $user, Product $product): bool
    {
        return $user->role === 'admin';
    }
}
```

Bước 4: Khóa các chức năng ở Controller (Bảo vệ Server-side)

Nếu chỉ ấn nút bấm trên giao diện, Hacker vẫn có thể gõ trực tiếp URL (ví dụ: <http://localhost:8000/products/1/edit>) hoặc dùng Postman để gửi request. Do đó, phải chặn ngay tại Controller.

Mở `app/Http/Controllers/ProductController.php` và thêm Gate vào đầu các hàm xử

lý dữ liệu:

```
namespace App\Http\Controllers;

use App\Models\Product;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Gate; // BẮT BUỘC IMPORT GATE

class ProductController extends Controller
{
    // index() và show() không cần chặn vì User cũng được xem

    public function create()
    {
        // Kiểm tra xem User có quyền tạo không? Nếu không tự động ném ra
        // lỗi 403
        Gate::authorize('create', Product::class);
        return view('products.create');
    }

    public function store(Request $request)
    {
        Gate::authorize('create', Product::class);

        // ... Code validate và lưu sản phẩm cũ của bạn ...
    }

    public function edit(Product $product) // Hoặc edit($id) tùy code cũ
    {
        // Truyền object $product vào để Policy kiểm tra
        Gate::authorize('update', $product);

        return view('products.edit', compact('product'));
    }

    public function update(Request $request, Product $product)
    {
        Gate::authorize('update', $product);

        // ... Code cập nhật sản phẩm cũ ...
    }
}
```

```
public function destroy(Product $product)
{
    Gate::authorize('delete', $product);

    // ... Code xóa sản phẩm cũ ...
}
}
```

Lưu ý: Từ Laravel 11, `Gate::authorize()` là cách chuẩn nhất. Ở các bản cũ hơn có thể dùng `$this->authorize()`.

Bước 5: Ẩn/Hiện phần tử Giao diện (Bảo vệ Client-side)

Mang lại trải nghiệm tốt cho người dùng: Nếu họ không có quyền thao tác, hệ thống không nên hiển thị các nút Thêm/Sửa/Xóa.

Mở file view chứa danh sách sản phẩm (ví dụ: `resources/views/products/index.blade.php`), sử dụng directive `@can` của Blade:

```
<!-- NÚT THÊM SẢN PHẨM -->
<!-- Sử dụng class Model vì hàm create không cần đối tượng cụ thể -->
@can('create', App\Models\Product::class)
    <a href="{{ route('products.create') }}" class="btn btn-success mb-3">+
Thêm Sản Phẩm Mới</a>
@endcan

<!-- BẢNG DANH SÁCH -->
<table class="table">
    <thead>
        <tr>
            <th>Tên sản phẩm</th>
            <th>Giá</th>
            <th>Hành động</th>
        </tr>
    </thead>
    <tbody>
        @foreach($products as $product)
            <tr>
                <td>{{ $product->name }}</td>
                <td>{{ number_format($product->price) }} VNĐ</td>
                <td>
```

```
<!-- Nút xem chi tiết: Ai cũng thấy -->
<a href="{{ route('products.show', $product->id) }}"
class="btn btn-info btn-sm">Xem</a>

<!-- NÚT SỬA SẢN PHẨM -->
@can('update', $product)
    <a href="{{ route('products.edit', $product->id)
}}" class="btn btn-warning btn-sm">Sửa</a>
@endcan

<!-- NÚT XÓA SẢN PHẨM -->
@can('delete', $product)
    <form action="{{ route('products.destroy',
$product->id) }}" method="POST" class="d-inline">
        @csrf
        @method('DELETE')
        <button type="submit" class="btn btn-danger
btn-sm" onclick="return confirm('Chắc chắn xóa?')">Xóa</button>
    </form>
@endcan
</td>
</tr>
@endforeach
</tbody>
</table>
```

Làm tương tự cho trang chi tiết `products.show.blade.php` nếu bạn có để nút Sửa/Xóa ở đó.

Bước 6: Bài tập Nâng cao: Chủ đề: Quyền sở hữu dữ liệu (Resource Ownership)

Hiện tại ở Bước 3, chúng ta đang thiết lập cứng nhắc: Chỉ Admin mới được Thêm/Sửa/Xóa, User thường chỉ được Xem. Tuy nhiên, trong thực tế (ví dụ: Shopee, Facebook), người dùng bình thường phải có quyền đăng sản phẩm/bài viết của chính họ, và họ có quyền Sửa/Xóa nội dung **do chính họ tạo ra**.

Yêu cầu bài tập: Hãy cấu hình lại ProductPolicy và giao diện để đáp ứng luật mới sau đây:

1. **Quyền Thêm mới (create):** Bất kỳ ai đã đăng nhập (cả admin và user) đều có quyền thêm sản phẩm mới.
2. **Quyền Sửa (update) và Xóa (delete):**

- Nếu là **Admin**: Được quyền Sửa/Xóa TẤT CẢ sản phẩm trên hệ thống.
- Nếu là **User thường**: CHỈ được quyền Sửa/Xóa những sản phẩm do **chính mình tạo ra** (Dựa vào kiến thức Lab 10: So sánh user_id của sản phẩm với id của user đang đăng nhập). Các sản phẩm của người khác thì chỉ được Xem.

Gợi ý thực hiện (Sinh viên tự suy nghĩ để code):

- Trong ProductPolicy, tại hàm update và delete, bạn cần kết hợp câu lệnh điều kiện logic `||` (HOẶC) và `==` (BẰNG).
- Ví dụ logic: Cho phép nếu "role của user là admin" HOẶC "id của user bằng user_id của product".
- Tại giao diện danh sách, tự động @can sẽ làm nhiệm vụ ẩn nút Sửa/Xóa ở những sản phẩm không thuộc về User đó.

BIỂU MẪU BÁO CÁO NỘI BÀI LAB 11

Sinh viên copy nội dung dưới đây, dán vào file Word MSSV_HoTen_Lab11.docx, điền thông tin và hình ảnh minh họa để nộp.

BÁO CÁO THỰC HÀNH LAB 11: PHÂN QUYỀN CHỨC NĂNG THEO VAI TRÒ

Thông tin sinh viên:

- **Họ và tên:**
- **Mã số sinh viên (MSSV):**

PHẦN 1: MÃ NGUỒN (SOURCE CODE)

1. Code ProductPolicy:

Yêu cầu dán mã nguồn dạng Text nội dung file app/Policies/ProductPolicy.php

[Sinh viên dán code vào đây]

2. Code bảo vệ Controller:

Yêu cầu dán mã nguồn dạng Text của hàm edit() và destroy() trong app/Http/Controllers/ProductController.php để minh chứng đã dùng Gate.

[Sinh viên dán code vào đây]

PHẦN 2: KỊCH BẢN KIỂM THỬ VỚI USER THƯỜNG (ROLE = 'user')

- **Hành động:** Đăng nhập bằng tài khoản có role là user. Vào trang /products.

- **Hình 1 (UI Test):** Chụp màn hình danh sách sản phẩm.
 - *Yêu cầu:* KHÔNG xuất hiện nút "Thêm Mới", "Sửa", "Xóa". Chỉ hiển thị nút "Xem". Trên thanh điều hướng phải thấy tên tài khoản User.

[Sinh viên chèn ảnh 1 vào đây]

- **Hình 2 (Security Test):** Gõ trực tiếp URL vào thanh địa chỉ: `http://localhost:8000/products/create` (hoặc `/edit` của 1 sản phẩm).
 - *Yêu cầu:* Chụp màn hình báo lỗi **403 THIS ACTION IS UNAUTHORIZED** để chứng minh Controller đã chặn thành công việc "vượt rào".

[Sinh viên chèn ảnh 2 vào đây]

PHẦN 3: KỊCH BẢN KIỂM THỬ VỚI ADMIN (ROLE = 'admin')

- **Hành động:** Đăng xuất, sau đó Đăng nhập bằng tài khoản có role là admin (đã chỉnh sửa trong Database). Vào trang `/products`.
- **Hình 3 (Admin UI Test):** Chụp màn hình danh sách sản phẩm.
 - *Yêu cầu:* Xuất hiện ĐẦY ĐỦ các nút chức năng: "Thêm Sản Phẩm Mới", "Sửa", "Xóa".

[Sinh viên chèn ảnh 3 vào đây]

PHẦN 4: QUYỀN SỞ HỮU DỮ LIỆU

1. Code *ProductPolicy* (Đã nâng cấp):

Yêu cầu dán mã nguồn dạng Text nội dung file `app/Policies/ProductPolicy.php` sau khi đã chỉnh sửa logic quyền sở hữu.

[Sinh viên dán code vào đây]

2. Kịch bản kiểm thử *Quyền sở hữu (Ownership Test)*:

- **Chuẩn bị:** Trong DB cần có ít nhất 2 sản phẩm:
 - Sản phẩm A: Do chính tài khoản User hiện tại tạo ra (trùng `user_id`).
 - Sản phẩm B: Do một người khác tạo ra (khác `user_id`).
- **Hành động:** Đăng nhập bằng tài khoản User **thường**. Vào trang `/products`.
- **Hình 4:** Chụp màn hình danh sách sản phẩm.
 - *Yêu cầu đạt:* Phải hiển thị nút "Thêm Mới" (vì User giờ đã được phép thêm). Ở

dòng Sản phẩm A phải có đủ nút Xem, Sửa, Xóa. Nhưng ở dòng Sản phẩm B, CHỈ HIỂN THỊ nút Xem (Nút Sửa/Xóa bị ẩn đi).

[Sinh viên chèn ảnh 4 vào đây]

- **Hình 5:** Vấn bằng tài khoản User thường đó, cố tình gõ trực tiếp URL để cố Sửa/Xóa Sản phẩm B (Ví dụ: <http://localhost:8000/products/2/edit>).
- *Yêu cầu đạt:* Hệ thống chặn lại và báo lỗi 403. Chụp màn hình báo lỗi.

[Sinh viên chèn ảnh 4.2 vào đây]